



TRAILHEAD
TECHNOLOGY PARTNERS

Mission-Critical Code

What NASA's Power of Ten
Can Teach Us



Jonathan "J." Tower



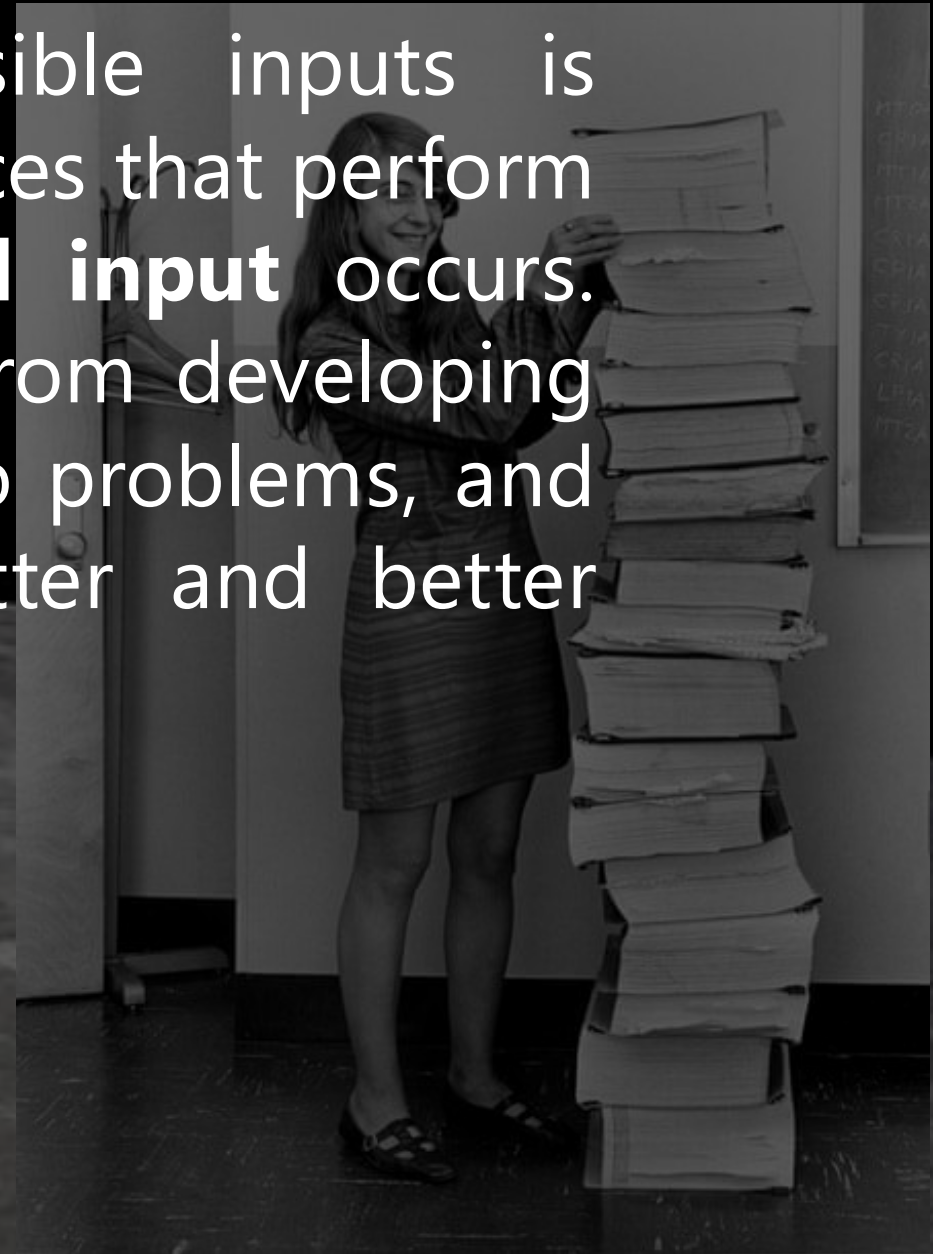




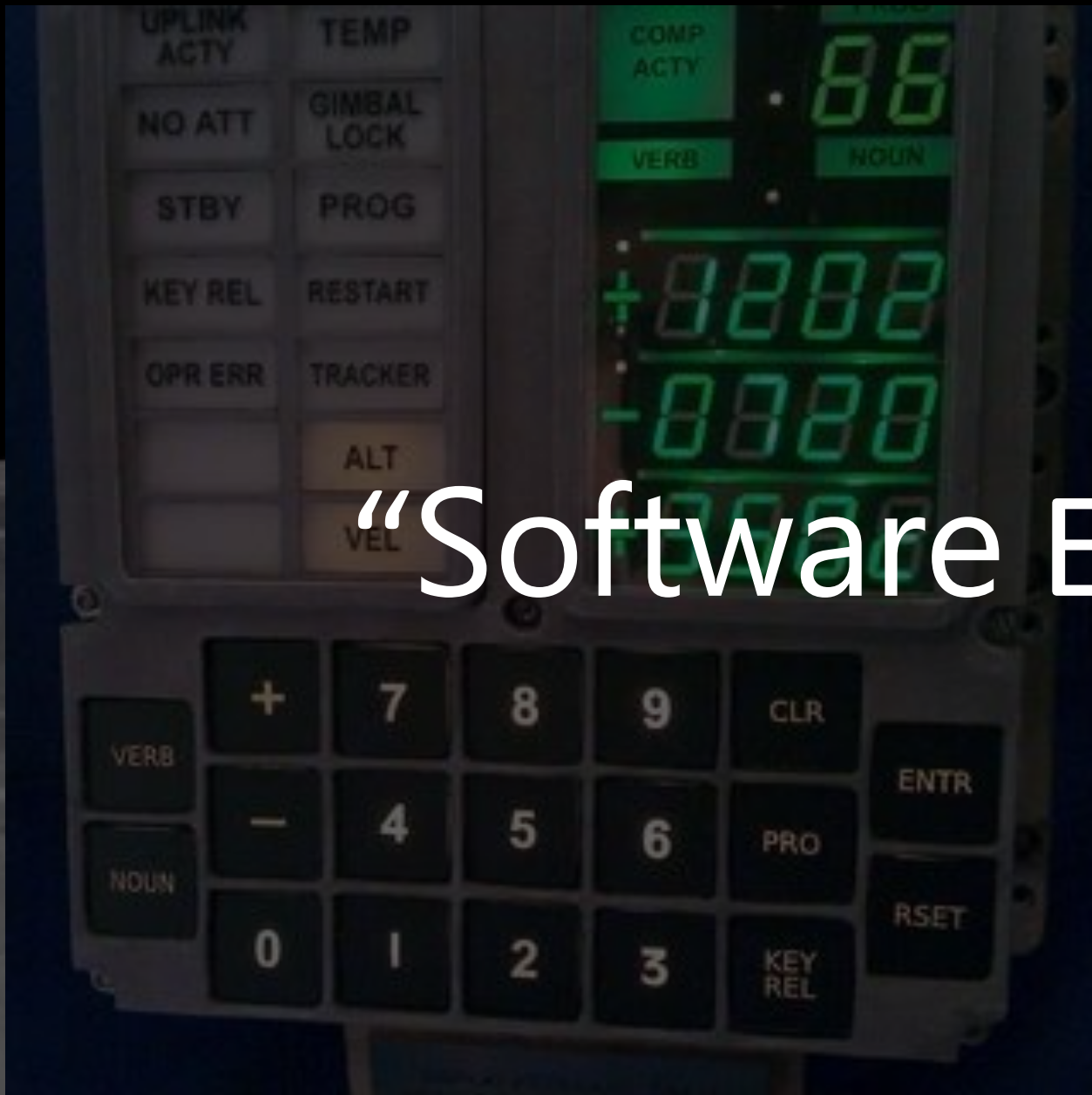
Margaret Hamilton

"...one's awareness of possible inputs is incomplete, leading to interfaces that perform poorly when an **unexpected input** occurs. The knowledge only comes from developing the software and running into problems, and incrementally evolving a better and better design."

- Margaret Hamilton



“Software Engineering”





Power of Ten



What we can learn from
NASA's **Power of Ten**
that will help us create
more reliable software

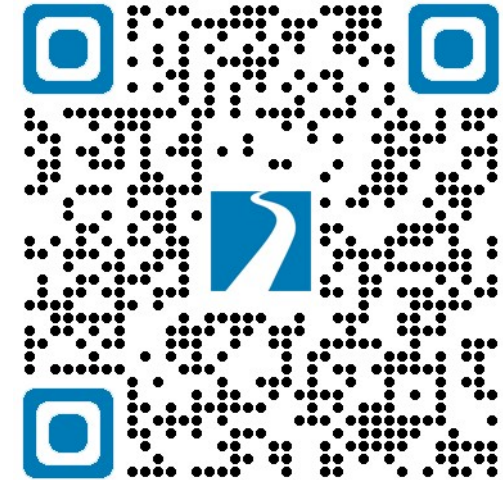
Jonathan "J." Tower

Principal Consultant & Partner



- 🏆 12x Microsoft MVP in .NET
- ⚙️ .NET Foundation Board Member-Elect
- 🎙️ Blue Blazes podcast host
- ✉️ jtower@trailheadtechnology.com
- 🌐 trailheadtechnology.com/blog
- ✂️ [jtowermi](#)
- 🌐 [Jonathan "J." Tower](#)

LET'S TALK



bit.ly/th-offer

github.com/trailheadtechnology/power-of-ten

What Is Mission-Critical Code?

From the Moon...to the Hospital



From Patients...to Money

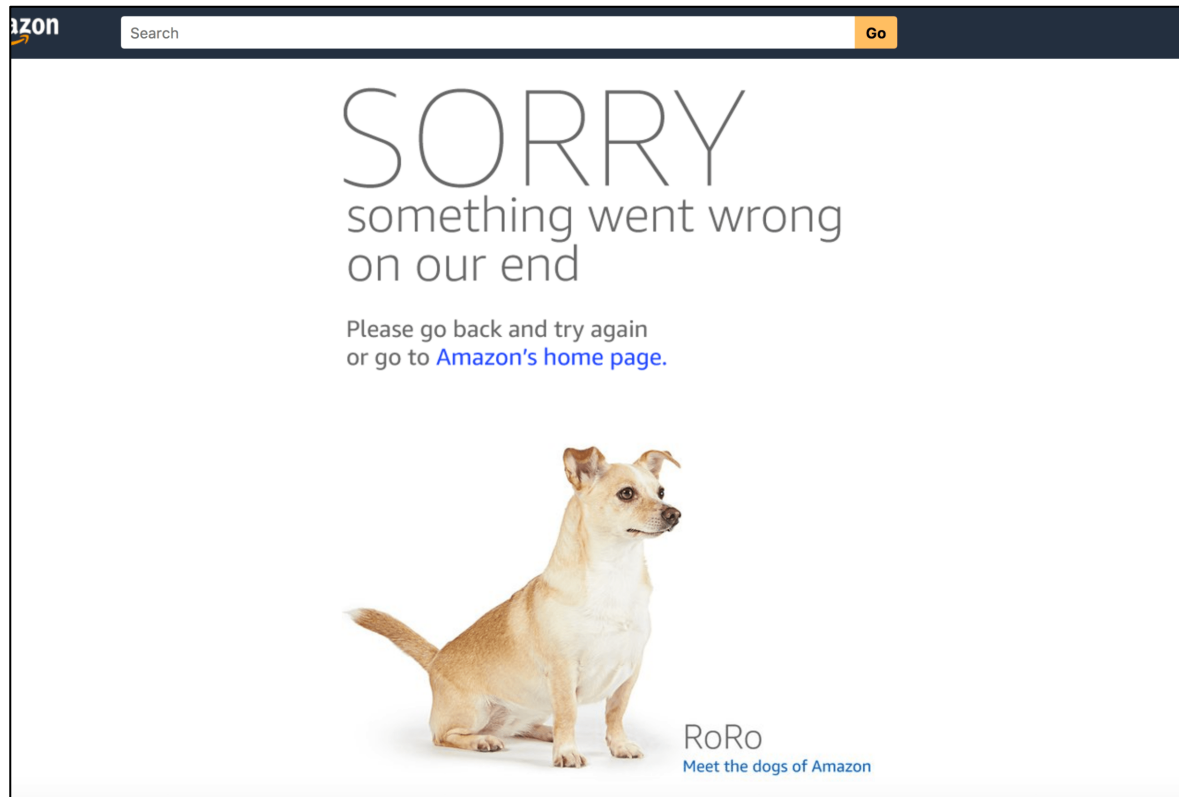


Knight

Knight Capital Group stock



From Trading...to Your Shopping Cart



report. All dates and times are Pacific Time (PST/PDT).

North America			South America	Europe	Asia Pacific	Contact Us
Recent Events		Details	RSS			
	Amazon Elastic Compute Cloud (N. Virginia)	Increased Error Rates more ▼				
	Amazon Elastic Load Balancing (N. Virginia)	Increased Error Rates more ▼				
	Amazon Relational Database Service (N. Virginia)	Increased Error Rates more ▼				
	Amazon Simple Storage Service (US Standard)	Increased Error Rates more ▼				
	Auto Scaling (N. Virginia)	Increased Error Rates more ▼				
	AWS Lambda (N. Virginia)	Increased Error Rates more ▼				
Remaining Services		Details	RSS			

Mission-Critical Code

✗ Where it runs



Mission-Critical Code

✗ Where it runs



✓ What happens when it fails

Mission-Critical Code

✗ Where it runs



✓ What happens when it fails

Software where failure causes **unacceptable harm** to:

Mission-Critical Code

✗ Where it runs



✓ What happens when it fails

Software where failure causes **unacceptable harm** to:

👤 Human Life

Mission-Critical Code

✗ Where it runs



✓ What happens when it fails

Software where failure causes **unacceptable harm** to:



Human Life

Safety

Mission-Critical Code

✗ Where it runs



✓ What happens when it fails

Software where failure causes **unacceptable harm** to:



Human Life



Safety



Money

Mission-Critical Code

✗ Where it runs



✓ What happens when it fails

Software where failure causes **unacceptable harm** to:

- 👤 Human Life
- 🚒 Safety
- 💰 Money
- 🤝 Trust

Mission-Critical Code

✗ Where it runs



✓ What happens when it fails

Software where failure causes **unacceptable harm** to:



Human Life



Safety



Money



Trust



Societal Systems

Mission-Critical Code

✗ Where it runs



✓ What happens when it fails

Software where failure causes **unacceptable harm** to:

- 👤 Human Life
- 🚒 Safety
- 💰 Money
- 🤝 Trust
- 🏛️ Societal Systems

And where recovery would be **too slow** or **too late**

Mission-Critical Code

✗ Where it runs



✓ What happens when it fails

Software where failure causes **unacceptable harm** to:



Human Life



Safety



Money



Trust



Societal Systems

And where recovery would be **too slow** or **too late**

Mission-Critical Code

✗ Where it runs



✓ What happens when it fails

Software where failure causes **unacceptable harm** to:



Human Life



Safety



Money



Trust



Societal Systems

We ALL write mission-critical code

And where recovery would be **too slow** or **too late**

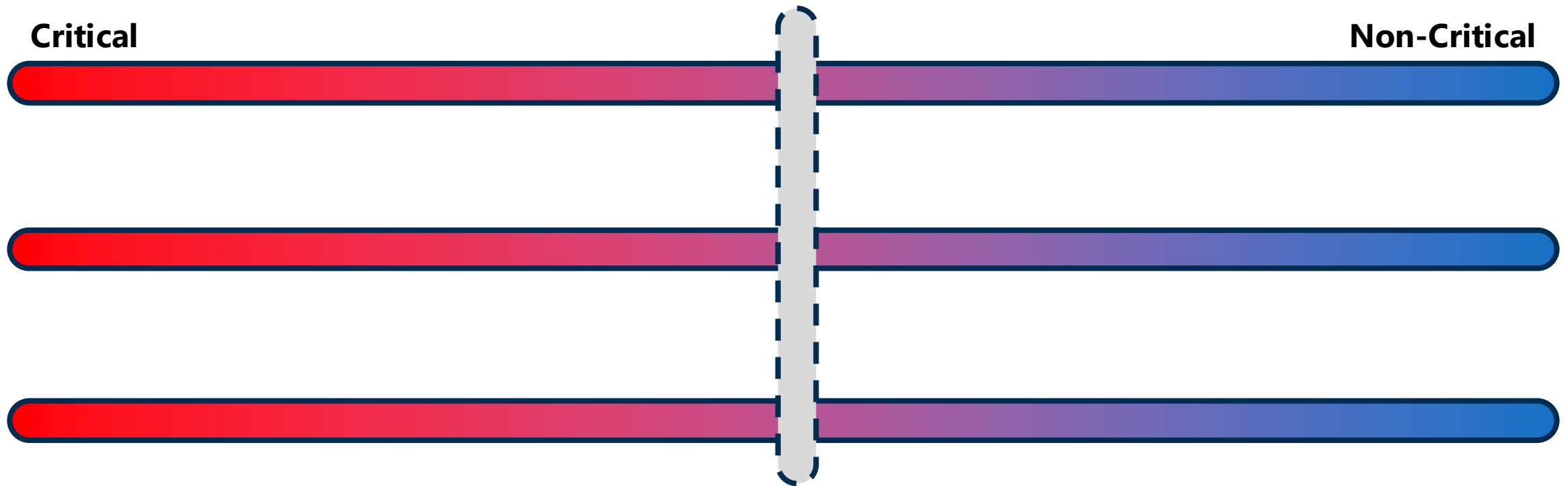
Setting The Right Level



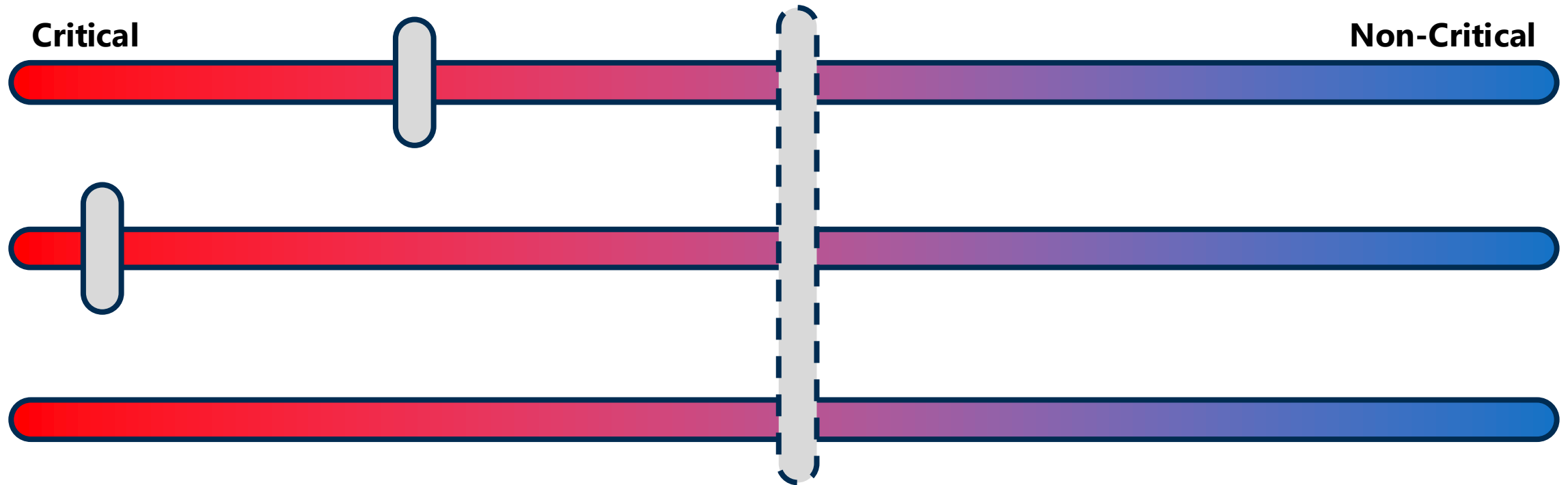
Setting The Right Level



Setting The Right Level



Setting The Right Level



History of Power of Ten



in spacecraft is increasing



Mission	Time
1	1000
2	1000
3	1000
4	1000
5	1000
6	1000
7	1000
8	1000
9	1000
10	1000



Gerard J. Holzmann



Gerard J. Holzmann

Patterns in Mission-Critical Software Failures



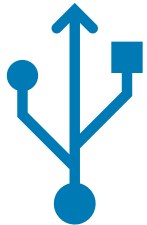
Excessive
Complexity



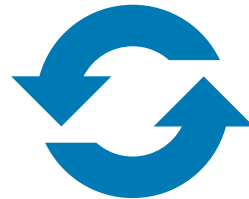
Hidden State



Inconsistent
Memory Usage
Patterns



Poorly Constrained
Interfaces



Unbounded
Behavior



Lack of Verifiable
Invariants

Power of Ten Was Meant to Produce



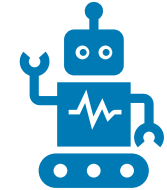
Predictable
Code



Testable
Code



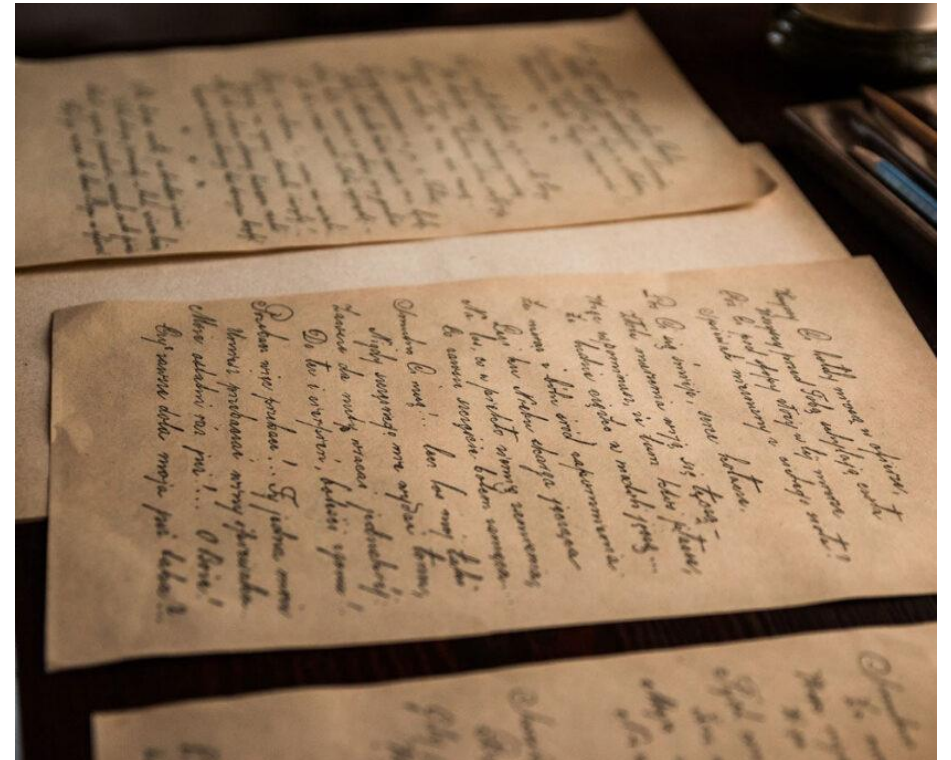
Auditable
Code



Verifiable by
Tools

The Original Power of Ten

1. Restrict all code to very simple control flow constructs
2. All loops must have a fixed upper-bound
3. Do not use dynamic memory allocation after initialization
4. No function should be longer than what can be printed on a single sheet of paper
5. Assertion density of the code should average to a minimum of two per function
6. Data objects must be declared at the smallest possible level of scope
7. Return value of non-void functions and validity of parameters must be checked
8. The use of the preprocessor must be limited
9. The use of pointers should be restricted
10. Zero warnings, most pedantic setting; daily static analysis



The Original Power of Ten



1. Restrict all control flow constructs
2. All loops must be upper-bound
3. Do not use dynamic allocation after initialization
4. No function should be longer than what can be printed on a single sheet of paper
5. Assertion density of the code should average to a minimum of two per function
6. Data objects must be declared at the smallest possible level of scope
7. Return values must be checked and validity of parameters must be verified
8. The use of type casting must be limited
9. The use of pointers must be restricted
10. Zero warning level; daily static analysis



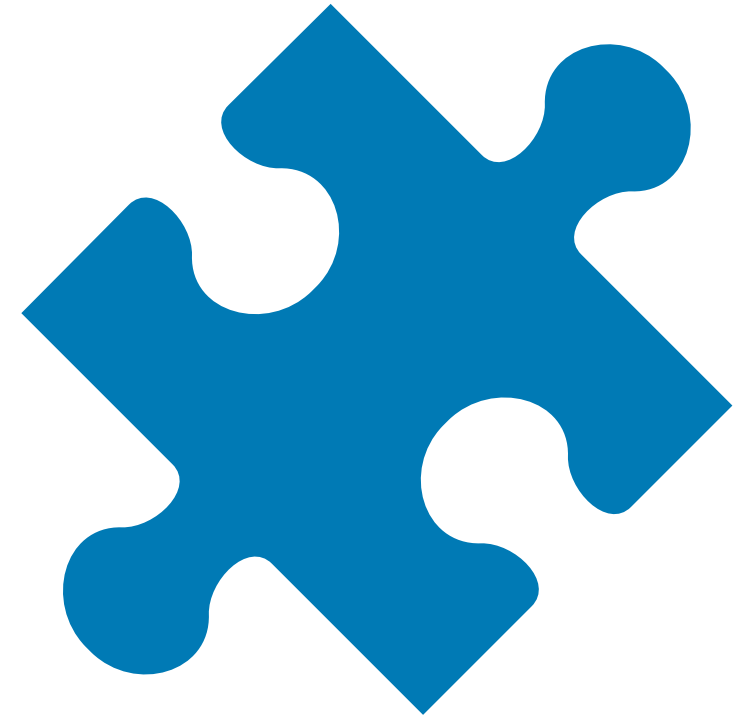
The Original Power of Ten



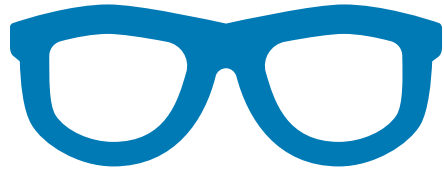
1. Restrict all control flow constructs
2. All loops must be upper-bound
3. Do not use dynamic allocation after initialization
4. No function should be longer than what can be printed on a single sheet of paper
5. Assertion density of the code should average to a minimum of two per function
6. Data objects must be declared at the smallest possible level of scope
7. Return values must be checked and validity of parameters must be verified
8. The use of the 'goto' statement must be limited
9. The use of pointers must be restricted
10. Zero warnings, no compiler flags; daily static analysis



1. Restrict all code
to very simple
control flow
constructs



Why Simple Control Flow?



Code Clarity



Verifiable Code

Anti-Pattern



```
public IEnumerable<Node> Flatten(Node root)
{
    yield return root;

    foreach (var child in root.Children)
    {
        foreach (var n in Flatten(child)) // X depth unknown
            yield return n;
    }
}
```

Pattern



```
public IEnumerable<Node> Flatten(Node root, CancellationToken ct)
{
    var stack = new Stack<Node>();
    var visited = new HashSet<Guid>(); // ✓ cycle guard
    stack.Push(root);

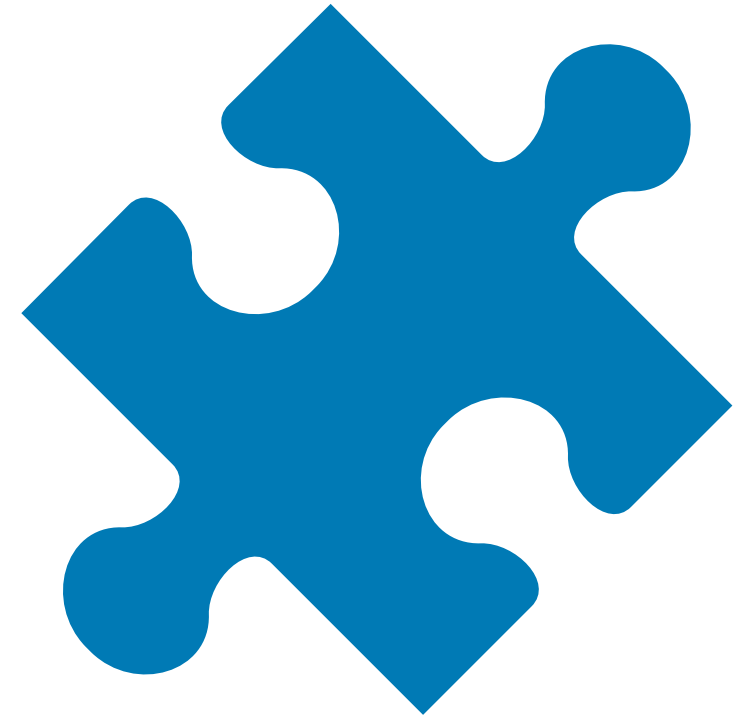
    while (stack.Count > 0) // ✓ explicit loop
    {
        ct.ThrowIfCancellationRequested();

        var node = stack.Pop();
        if (!visited.Add(node.Id)) // ✓ prevents cycles
            continue;

        yield return node;

        // ✓ you control order, bounds, and can add depth limits
        foreach (var child in node.Children) stack.Push(child);
    }
}
```

2. All loops must
have a fixed
upper-bound



Why Bounded Loops?



Runaway
Code



Memory
Overflows

Anti-Pattern



```
public async Task WaitForReadyAsync(CancellationTokentoken ct)
{
    while (true) // ✗ no upper bound
    {
        var status = await _client.GetStatusAsync(ct);

        if (status == "Ready") break; // ✗ assumes "Ready" will happen

        await Task.Delay(500, ct); // ✗ can run forever if status never flips
    }
}
```

Pattern



```
public async Task WaitForReadyAsync(CancellationTokentoken ct)
{
    const int maxAttempts = 20;

    for (var attempt = 1; attempt <= maxAttempts; attempt++) //  fixed bound
    {
        var status = await _client.GetStatusAsync(ct);
        if (status == "Ready") return;

        await Task.Delay(500, ct);
    }

    throw new TimeoutException("Service never became Ready within maxAttempts.");
}
```

Anti-Pattern



```
public async Task<int> CountMatchesAsync(string path, string term, CancellationToken
ct)
{
    var count = 0;

    using var reader = File.OpenText(path);

    while (await reader.ReadLineAsync(ct) is { } line) // X unbounded work
    {
        if (line.Contains(term, StringComparison.OrdinalIgnoreCase))
            count++;
    }

    return count;
}
```

Pattern



```
public async Task<int> CountMatchesAsync(string path, string term, CancellationToken ct)
{
    const int maxLines = 50_000; // ✓ upper bound
    var count = 0;
    var linesRead = 0;

    using var reader = File.OpenText(path);

    while (await reader.ReadLineAsync(ct) is { } line)
    {
        if (++linesRead > maxLines) return -1; // ✓ upper bound reached -> error
        if (line.Contains(term, StringComparison.OrdinalIgnoreCase)) count++;
    }

    return count;
}
```

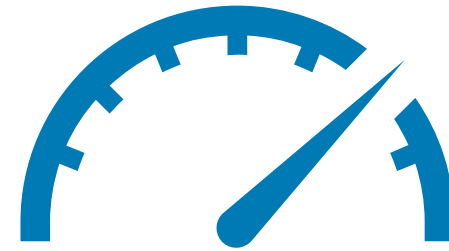
3. Do not use
dynamic memory
allocation after
initialization



Why Limit Resource Usage?



Memory Leaks



Performance

Anti-Pattern



```
public async Task<IReadOnlyList<OrderDto>> GetOrdersLastDayAsync()
{
    using var client = new HttpClient(); // ✗ per-call client (connection churn)

    var since = DateTime.UtcNow.AddDays(-1);
    var orders = await _db.Orders
        .Where(o => o.CreatedAtUtc >= since)
        .OrderByDescending(o => o.CreatedAtUtc)
        .ToListAsync(); // ✗ time-bound, not volume-bound; loads all

    var results = new List<OrderDto>(); // ✗ grows unbounded

    foreach (var order in orders)
    {
        var response = await client.GetAsync($"/api/orders/{order.Id}/total"); // ✗ no timeout/cancel; fan-out
        var price = await response.Content.ReadFromJsonAsync<decimal>(); // ✗ no success check

        results.Add(new OrderDto(order.Id, order.CustomerId, price));
    }

    return results;
}
```

Pattern



```
public async Task<IReadOnlyList<OrderDto>> GetOrdersLastDayAsync(int limit, CancellationToken ct)
{
    limit = Math.Clamp(limit, 1, 200); // ✅ bound work
    var client = _httpClientFactory.CreateClient("pricing"); // ✅ pooled client

    var since = DateTime.UtcNow.AddDays(-1);
    var orders = await _db.Orders.Where(o => o.CreatedAtUtc >= since).OrderByDescending(o => o.CreatedAtUtc)
        .Select(o => new { o.Id, o.CustomerId })
        .Take(limit) // ✅ volume bound
        .ToListAsync(ct); // ✅ cancellable

    var results = new List<OrderDto>(orders.Count); // ✅ bounded allocation

    foreach (var o in orders)
    {
        using var response = await client.GetAsync($""/api/orders/{o.Id}/total", ct); // ✅ cancellable
        response.EnsureSuccessStatusCode(); // ✅ fail fast

        var price = await response.Content.ReadFromJsonAsync<decimal>(ct);
        results.Add(new OrderDto(o.Id, o.CustomerId, price));
    }
    return results;
}
```

Anti-Pattern



```
static readonly Dictionary<string, object> Cache = new();

public object Get(string key)
{
    if (!Cache.TryGetValue(key, out var v))
        Cache[key] = v = Load(key); // grows forever
    return v;
}
```

Pattern



```
private readonly IMemoryCache _cache =
    new MemoryCache(new MemoryCacheOptions
    {
        SizeLimit = 10_000 // ✓ fixed upper bound
    });

public object Get(string key)
{
    return _cache.GetOrCreate(key, entry =>
    {
        entry.Size = 1; // ✓ every entry counts toward the limit
        entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(5);

        return Load(key);
    });
}
```

Stack vs Heap

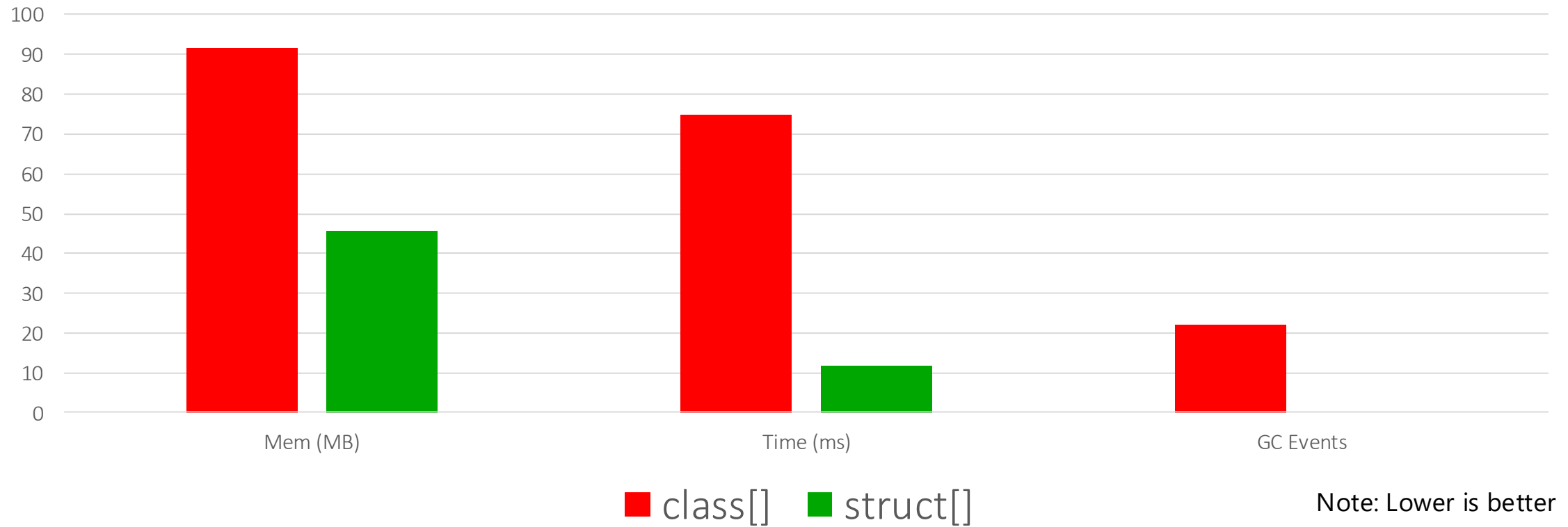
// stores on the heap

```
public record class TelemetryEvent(  
    int SensorId,  
    long TimestampUnixMs,  
    double Value);
```

// stores on the stack

```
public record struct TelemetryEvent(  
    int SensorId,  
    long TimestampUnixMs,  
    double Value);
```

Stack vs Heap



4. No function should be longer than what can be printed on a single sheet of paper



Why Short Functions?

 0 – 50 lines

Simple, low cognitive load, low risk of bugs

 50 – 150 lines ← **“Sweet Spot”**

Lowest change frequency, manageable complexity

 150 – 200 lines

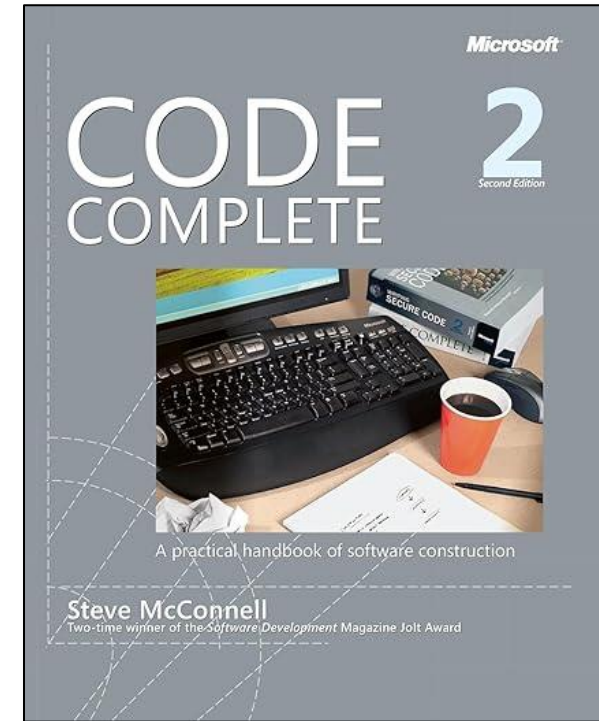
Acceptable for complex algorithms

 200 – 500 lines

Risky

 500+ lines

Error rate proportional to size



Code Complete, 2nd Edition
by Steve McConnell

Anti-Pattern

The "God" Function

```
public async Task ProcessOrderAsync(Guid orderId)
{
    var order = await _db.Orders.Include(o => o.Items).SingleAsync(o => o.Id == orderId);

    if (order.Status == OrderStatus.Closed) return;
    if (order.Items.Count == 0) throw new InvalidOperationException("Empty order");

    decimal total = 0;
    foreach (var item in order.Items)
    {
        total += item.Price * item.Quantity;
        if (item.DiscountCode != null) total -= CalculateDiscount(item);
    }

    var tax = _taxService.Calculate(order.Address, total);
    total += tax;
    await _paymentGateway.ChargeAsync(order.CustomerId, total);

    order.Status = OrderStatus.Closed;
    order.ClosedAt = DateTime.UtcNow;

    await _db.SaveChangesAsync();
    await _email.SendReceiptAsync(order);
}
```

Anti-Pattern

The "God" Function



```
public async Task ProcessOrderAsync(Guid orderId)
{
    var order = await _db.Orders.Include(o => o.Items).SingleAsync(o => o.Id == orderId);

    if (order.Status == OrderStatus.Closed) return;
    if (order.Items.Count == 0) throw new InvalidOperationException("Empty order");

    decimal total = 0;
    foreach (var item in order.Items)
    {
        total += item.Price * item.Quantity;
        if (item.DiscountCode != null) total -= CalculateDiscount(item);
    }
}
```

● Multiple responsibilities

- Validation
- Pricing
- Tax
- Payment
- Persistence
- Notifications

```
var tax = _taxService.Calculate(order.Address, total);
total += tax;
await _paymentGateway.ChargeAsync(order.CustomerId, total);

order.Status = OrderStatus.Closed;
order.ClosedAt = DateTime.UtcNow;

await _db.SaveChangesAsync();
await _email.SendReceiptAsync(order);
}
```

Anti-Pattern

The "God" Function



```
public async Task ProcessOrderAsync(Guid orderId)
{
    var order = await _db.Orders.Include(o => o.Items).SingleAsync(o => o.Id == orderId);

    if (order.Status == OrderStatus.Closed) return;
    if (order.Items.Count == 0) throw new InvalidOperationException("Empty order");

    decimal total = 0;
    foreach (var item in order.Items)
    {
        total += item.Price * item.Quantity;
        if (item.DiscountCode != null) total -= CalculateDiscount(item);
    }
}
```

- Multiple responsibilities
 - Validation
 - Pricing
 - Tax
 - Payment
 - Persistence
 - Notifications

```
var tax = _taxService.Calculate(order.Address, total);
total += tax;
await _paymentGateway.ChargeAsync(order.CustomerId, total);

order.Status = OrderStatus.Closed;
order.ClosedAt = DateTime.UtcNow;

await _db.SaveChangesAsync();
await _email.SendReceiptAsync(order);
}
```

- Hidden decision logic
- Hard to test
- Partial failure risk
- Any change = high blast radius

Pattern

The "Orchestrator" Function



```
public async Task CloseOrderAsync(Guid orderId)
{
    var order = await LoadOrder(orderId);
    EnsureOrderIsClosable(order);

    var pricing = CalculatePricing(order);
    await ChargeCustomer(order.CustomerId, pricing.Total);

    MarkOrderClosed(order);
    await Save(order);
    await SendReceipt(order, pricing);
}
```

Pattern

The "Orchestrator" Function



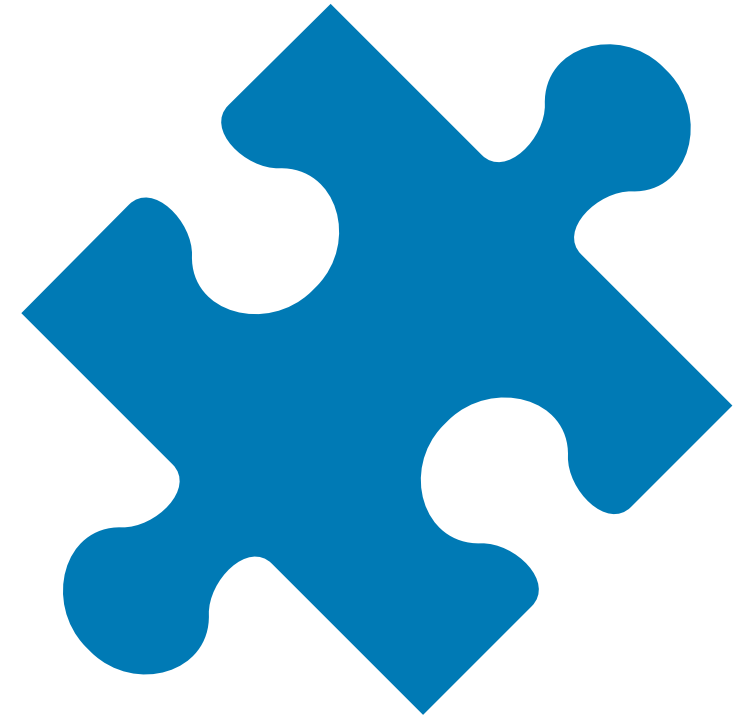
```
public async Task CloseOrderAsync(Guid orderId)
{
    var order = await LoadOrder(orderId);
    EnsureOrderIsClosable(order);

    var pricing = CalculatePricing(order);
    await ChargeCustomer(order.CustomerId, pricing.Total);

    MarkOrderClosed(order);
    await Save(order);
    await SendReceipt(order, pricing);
}
```

- Each function has one job
- Readable top-down
- Easy to test in isolation
- Failures are localized
- Predictable execution

5. Assertion density of the code should average to a minimum of two per function

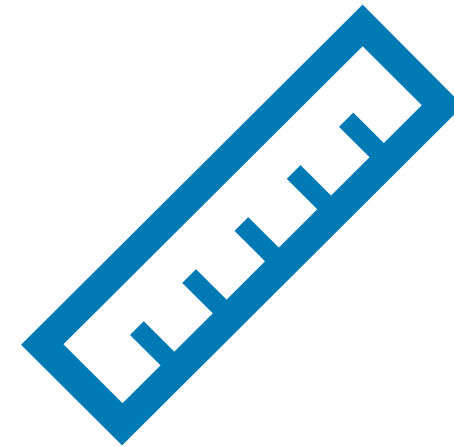


Why Assertions?

Assertions are **executable contracts** for states that “should never happen.”



Unit Tests: 1
Defect Per
10-100 Lines



More Caught
With More
Assertions

Anti-Pattern



```
public async Task CloseOrderAsync(Guid orderId)
{
    var order = await _db.Orders.FindAsync(orderId);

    if (order == null)
    {
        _logger.LogWarning("Order not found: {OrderId}", orderId); // ✗ logs
        return; // ✗ caller thinks success
    }

    // ✗ logs but continues anyway
    if (order.Total <= 0)
        _logger.LogWarning("Suspicious total: {Total}", order.Total);

    await _payments.ChargeAsync(order.CustomerId, order.Total); // ✗ may charge 0 or invalid

    order.Status = OrderStatus.Closed; // ✗ invalid transition possible
    await _db.SaveChangesAsync();
}
```

Pattern



```
public async Task<bool> TryCloseOrderAsync(Guid orderId, CancellationToken ct)
{
    if (orderId == Guid.Empty) return false; // ✅ precondition

    var order = await _db.Orders.FindAsync([orderId], ct);
    if (order is null) return false; // ✅ explicit failure

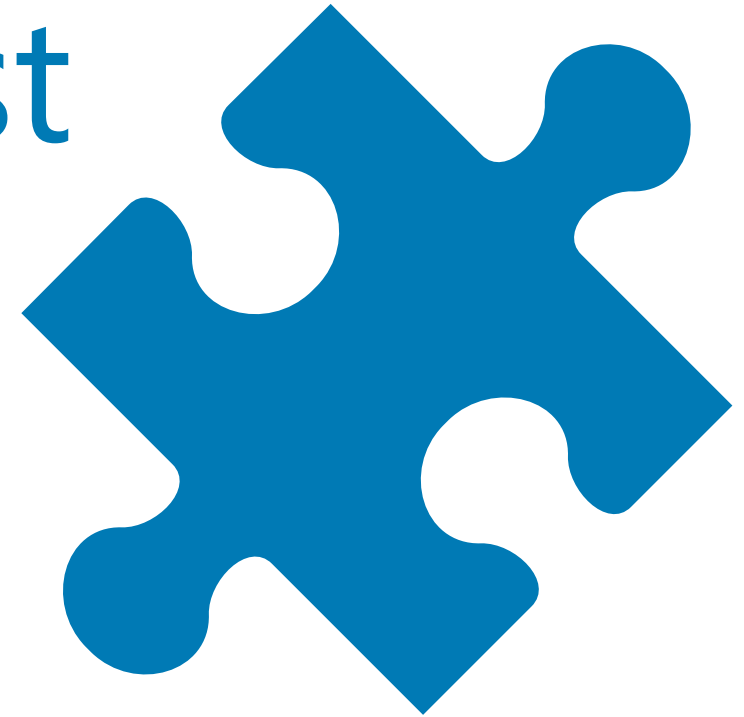
    if (order.Status == OrderStatus.Closed) return true; // ✅ idempotent ok
    if (order.Total <= 0) return false; // ✅ invariant before side effects

    var receipt = await _payments.TryChargeAsync(order.CustomerId, order.Total, ct);
    if (receipt is null) return false; // ✅ postcondition on dependency call

    order.Close(receipt.Id); // ✅ single place for domain invariant
    await _db.SaveChangesAsync(ct);

    return true; // ✅ indicates actual success
}
```

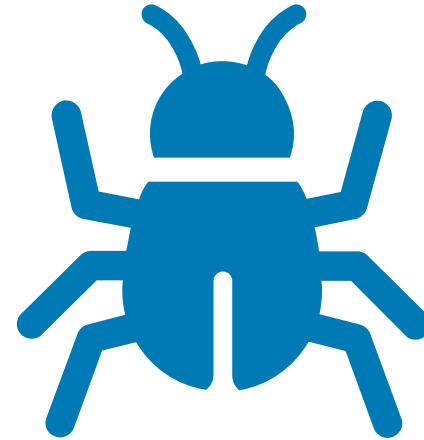
6. Data objects must be declared at the smallest possible level of scope



Why Limit Data Scope?



Can't Corrupt
What Can't
You See



Easier To Debug
Variables With
Limited Scope

Anti-Pattern



```
public sealed class PricingService
{
    private decimal _price; // ✗ reused across iterations

    public decimal SumPrices(IEnumerable<int> productIds)
    {
        decimal total = 0;
        foreach (var id in productIds)
        {
            _price = _repo.GetPrice(id); // ✗ state lives beyond iteration
            if (_price < 0) _logger.LogWarning("Negative price: {Id}", id);

            total += _price;
        }
        return total;
    }
}
```

Pattern



```
public sealed class PricingService
{
    public decimal SumPrices(IEnumerable<int> productIds)
    {
        decimal total = 0;
        foreach (var id in productIds)
        {
            var price = _repo.GetPrice(id); // ✓ per-iteration scope
            if (price < 0) _logger.LogWarning("Negative price: {Id}", id);

            total += price;
        }
        return total;
    }
}
```

Anti-Pattern



```
public decimal CalculateTotal(Order order)
{
    decimal value = order.Subtotal; // X starts as subtotal

    value = ApplyDiscounts(order, value); // X now discounted subtotal
    value = AddTax(order, value);          // X now taxed total
    value = Round(value);                  // X now rounded final

    return value; // X debugging: what is "value" right now?
}
```

Pattern

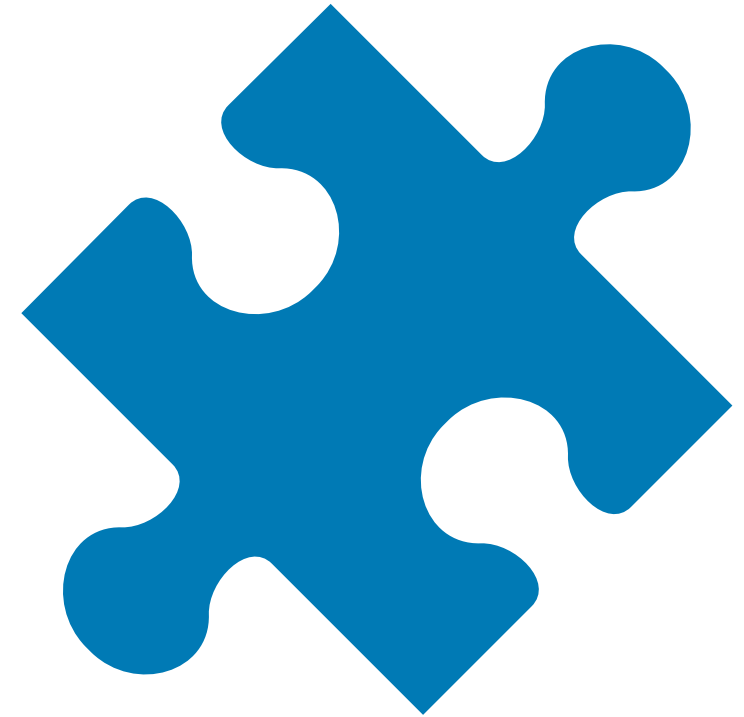


```
public decimal CalculateTotal(Order order)
{
    var subtotal = order.Subtotal;
    var discounted = ApplyDiscounts(order, subtotal);
    var taxed = AddTax(order, discounted);
    var finalTotal = Round(taxed);

    return finalTotal;
}
```

// ✓ clear meaning
// ✓ clear meaning
// ✓ clear meaning
// ✓ clear meaning

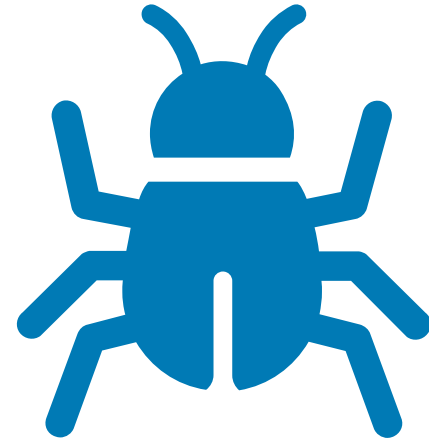
7. Return value of non-void functions and validity of parameters must be checked



Why Return Success?



Failures Are
Data Too



Propagate Errors
Early

Anti-Pattern



```
public static int ReadPort(string input)
{
    int.TryParse(input, out var port); // ✗ return value ignored
    return port; // ✗ port may be 0 on failure
}

//NOTE: Def of int.TryParse
public static bool TryParse(string? s, IFormatProvider? provider, out int result);
```

Pattern



```
public static int ReadPort(string input)
{
    if (!int.TryParse(input, out var port)) //  explicitly checking success flag
        throw new ArgumentException("Invalid port.");

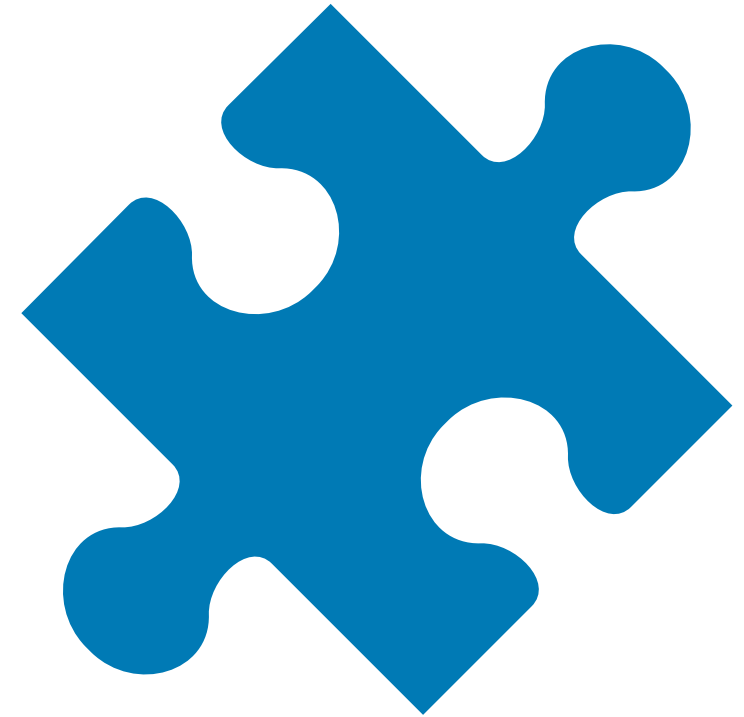
    return port;
}
```

Pattern



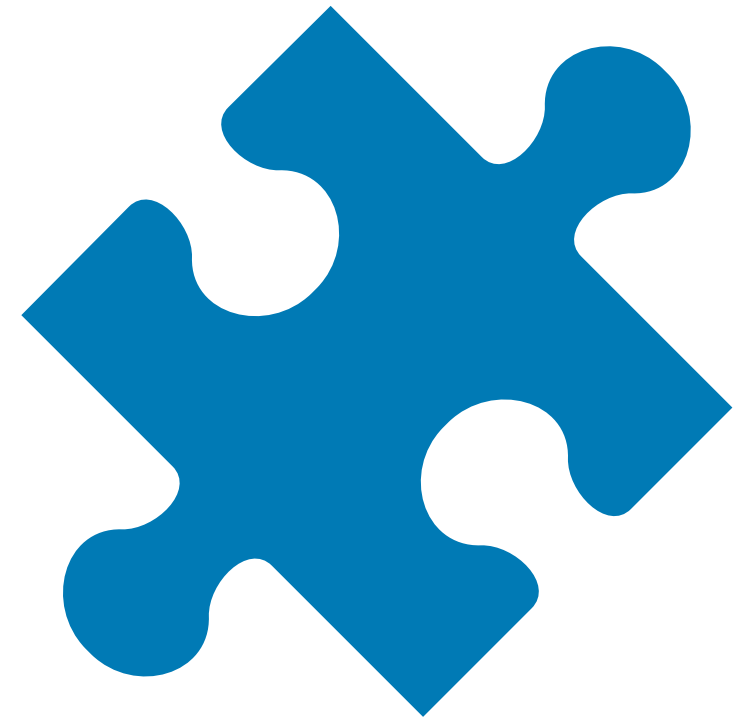
```
public static int ReadPortOrDefault(string input)
{
    _ = int.TryParse(input, out var port); // ✓ explicitly ignoring success flag
    return port == 0 ? 443 : port;         // ✓ fallback (example default)
}
```

8. The use of the
preprocessor must
be limited

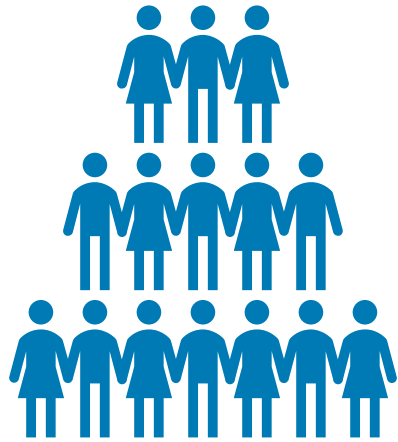


~~8. The use of the preprocessor must be limited~~

“Limit the use of metaprogramming”



Why Limit Metaprogramming?



Many Variants
To Test



Complex To
Maintain

Why Limit Metaprogramming?

$$2^{10} = 1,024$$

possible variations to test

Anti-Pattern



```
public static decimal Total(decimal subtotal)
{
    #if FEATURE_DISCOUNTS
        subtotal = ApplyDiscounts(subtotal);
    #endif

    #if FEATURE_TAX
        subtotal = AddTax(subtotal);
    #endif

    #if FEATURE_SURCHARGE
        subtotal = AddSurcharge(subtotal);
    #endif

    #if FEATURE_ROUNDING
        subtotal = Round(subtotal);
    #endif

    return subtotal;
}
```

Pattern



```
public sealed record CheckoutFeatures(bool Discounts, bool Tax, bool Surcharge, bool Rounding);

public static class Checkout
{
    public static decimal Total(decimal subtotal, CheckoutFeatures f)
    {
        if (f.Discounts) subtotal = ApplyDiscounts(subtotal);
        if (f.Tax)         subtotal = AddTax(subtotal);
        if (f.Surcharge)   subtotal = AddSurcharge(subtotal);
        if (f.Rounding)    subtotal = Round(subtotal);

        return subtotal;
    }
}
```

9. Use of pointers should be restricted

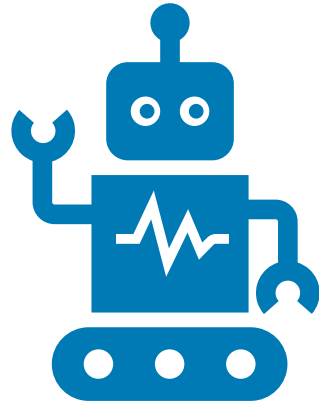


~~9. Use of pointers should be restricted~~

“Limit indirection and dynamic dispatch”



Why Limit Indirection & Dynamic Dispatch?



Breaks Static
Analysis



Complex To
Maintain

Anti-Pattern



```
public sealed class CommandBus
{
    private readonly IServiceProvider _sp;

    public CommandBus(IServiceProvider sp) => _sp = sp;

    public Task DispatchAsync(string commandName, object payload)
    {
        var handlerTypeName = $"MyApp.Commands.{commandName}Handler"; // ✗ “stringly”-typed
        var handlerType = Type.GetType(handlerTypeName)!;                // ✗ null at runtime

        var handler = _sp.GetRequiredService(handlerType);              // ✗ unknown wiring

        var method = handlerType.GetMethod("HandleAsync")!;              // ✗ signature mismatch at runtime
        return (Task)method.Invoke(handler, new[] { payload })!;         // ✗ reflection, no compile-time
    }
}
```

safety

Pattern



```
public interface ICommandHandler<in TCommand>
{
    Task HandleAsync(TCommand command, CancellationToken ct);
}

public sealed class CommandBus
{
    private readonly IServiceProvider _sp;

    public CommandBus(IServiceProvider sp) => _sp = sp;

    public Task DispatchAsync<TCommand>(TCommand command, CancellationToken ct)
    {
        var handler = _sp.GetRequiredService<ICommandHandler<TCommand>>(); // ✅ typed
        return handler.HandleAsync(command, ct); // ✅ direct call
    }
}
```

Anti-Pattern



```
public async Task ProcessPaymentAsync(Order order, Func<Task> afterCharge)
{
    await _payments.ChargeAsync(order.CustomerId, order.Total); //  did the thing

    await afterCharge(); //  arbitrary code runs here (can throw, can call back into us)

    order.Status = OrderStatus.Paid; //  might never run if callback throws
    await _db.SaveChangesAsync();
}
```

Pattern



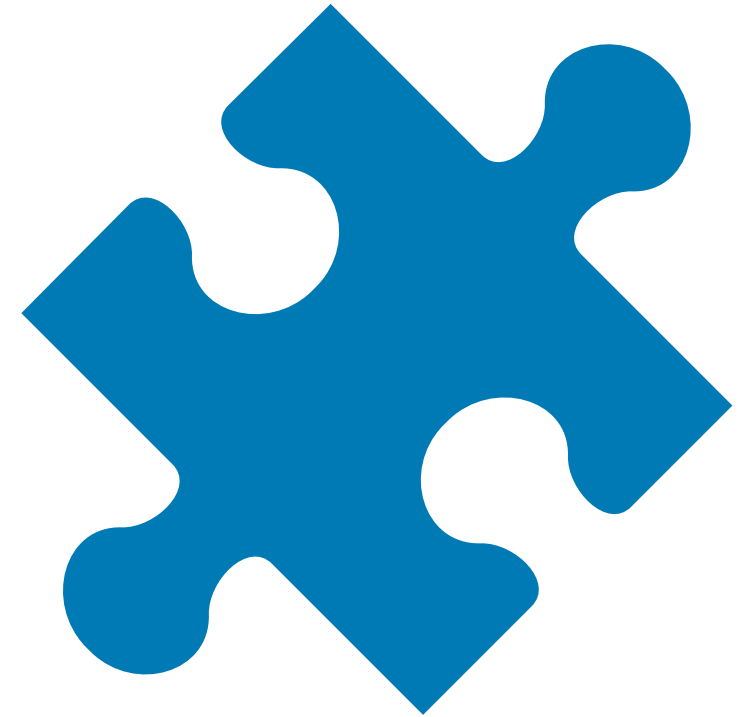
```
public interface IAfterChargeStep { Task RunAsync(Order order, CancellationToken ct); }

public PaymentService(IAfterChargeStep afterChargeStep)
{
    _afterChargeStep = afterChargeStep; // ✅ single, explicit dependency
}

public async Task ProcessPaymentAsync(Order order, IAfterChargeStep step, CancellationToken ct)
{
    await _payments.ChargeAsync(order.CustomerId, order.Total, ct);
    try {
        await step.RunAsync(order, ct); // ✅ known interface + testable
    } catch (Exception) {}

    order.Status = OrderStatus.Paid;
    await _db.SaveChangesAsync(ct);
}
```

10. Zero warnings,
most pedantic
setting; daily static
analysis



Why Warnings and Static Analysis?



Warnings Are
a Quality Gate



If Analyzer is
Confused, Humans
Will Be Too

Anti-Pattern



`#nullable disable // ✗ turns off a whole class of correctness checks`

```
public string GetEmail(User user)
```

```
{
```

```
#pragma warning disable CS8602 // ✗ suppress instead of fixing
```

```
    return user.Profile.Email.ToLower(); // ✗ possible null deref
```

```
#pragma warning restore CS8602
```

```
}
```

Pattern



```
public static string GetEmail(User user)
{
    ArgumentNullException.ThrowIfNull(user);
    ArgumentNullException.ThrowIfNull(user.Profile);

    var email = user.Profile.Email;
    if (string.IsNullOrEmpty(email))
        throw new InvalidOperationException("User has no email.");

    return email.ToLowerInvariant();
}
```

Anti-Pattern



Error List		
Entire Solution		
2461 Errors 0 of 9514 Warnings 0 of 6530 Messages		
Code	Description	
CS0103	The name 'InitializeComponent' does not exist in the current context	SE
CS0103	The name 'InitializeComponent' does not exist in the current context	SE
CS0103	The name 'InitializeComponent' does not exist in the current context	SE
CS0103	The name 'InitializeComponent' does not exist in the current context	SE
CS0103	The name 'loadingproject' does not exist in the current context	SE
CS0103	The name 'loadingdesign' does not exist in the current context	SE
CS0103	The name 'loadinganchors' does not exist in the current context	SE
CS0103	The name 'loadingresults' does not exist in the current context	SE
CS0103	The name 'loadingmodels' does not exist in the current context	SE
CS0103	The name 'finishingup' does not exist in the current context	SE

Don't Just
Turn Off or
Ignore
Warnings

Pattern



Summing Up

1. Code doesn't have to be on a **rocket** to be **mission-critical**
2. Applying rules like NASA's **Power of Ten** makes your code more **robust** and **reliable**

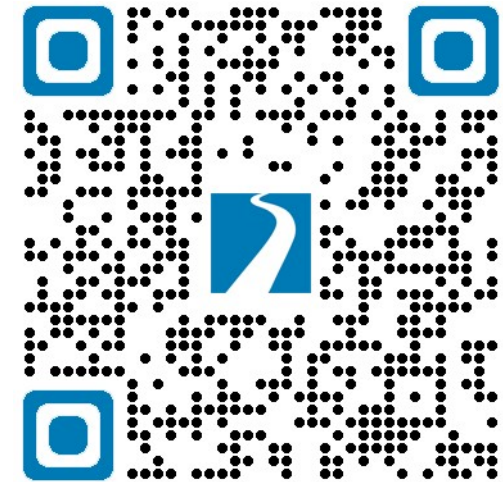


Thanks! Questions?

Jonathan "J." Tower

- 🏆 12x Microsoft MVP in .NET
- ⚙️ .NET Foundation Board Member
- 🎙️ Blue Blazes podcast host
- ✉️ jtower@trailheadtechnology.com
- 🌐 trailheadtechnology.com/blog
- ✂️ [jtowermi](#)
- 📘 Jonathan "J." Tower

LET'S TALK



bit.ly/th-offer

github.com/trailheadtechnology/power-of-ten